

SAVCNG ExcelDNA - Documentación Técnica

Tabla de Contenidos

- 1. Descripción General
 - 2. Arquitectura del Proyecto
 - 3. Componentes Principales
 - 4. Sistema de Validaciones
 - 5. Flujo de Trabajo
 - 6. Especificaciones Técnicas
 - 7. Guía de Desarrollo
-

Descripción General

SAVCNG ExcelDNA es un complemento (Add-in) de Microsoft Excel desarrollado en C# (.NET Framework 4.8) que proporciona un sistema avanzado de validación de datos para archivos de censo de gobierno.

El proyecto utiliza **ExcelDNA** como framework de integración con Excel, permitiendo crear un menú personalizado en la cinta de opciones (Ribbon) de Excel desde el cual se pueden cargar archivos de censo y aplicar múltiples validaciones de integridad de datos.

Propósito Principal

- Garantizar la calidad e integridad de los datos en archivos de censo
- Proporcionar una interfaz gráfica intuitiva para validación de datos
- Detectar y resaltar errores de formato y contenido
- Facilitar listas desplegables (Data Validation) personalizadas

Tecnologías Utilizadas

- **Lenguaje:** C# 7.3
 - **Framework:** .NET Framework 4.8
 - **Librería de Interop:** Microsoft.Office.Interop.Excel
 - **Plugin Framework:** ExcelDNA
 - **UI:** Windows Forms (.NET Framework)
-

Arquitectura del Proyecto

SAVCNG_ExcelDNA/	
├─ MenuCenso.cs	(Punto de entrada y menú)
├─ FrmValidaciones.cs	(Lógica principal de validaciones)
└─ FrmValidaciones.Designer.cs	(Diseño de formulario)

```
| Properties/
|   | AssemblyInfo.cs      (Metadatos del ensamblado)
|   | obj/                 (Artefactos compilados)
```

Patrón Arquitectónico

- **Model:** Las colecciones y diccionarios almacenan datos de validación
- **View:** Windows Forms (FrmValidaciones) para la interfaz
- **Controller:** Métodos de eventos que procesan interacciones del usuario

Componentes Principales

1. MenuCenso.cs - Punto de Entrada

Responsable de la integración con Excel y la interfaz de usuario.

Características Principales:

```
public class MenuCenso : ExcelRibbon
{
    // Define un menú personalizado en la cinta de Excel
    public override string GetCustomUI(string RibbonID)
}
```

Funcionalidades:

- **XML Ribbon:** Define la estructura del menú en la pestaña "SAVCNG - Censo"
- **Diálogo de Archivo:** Permite seleccionar archivos de censo (.xlsx, .xlsm, .xlsb)
- **Gestión de Workbook:** Abre el archivo y lo pasa a la ventana de validaciones
- **Manejo de Errores:** Captura excepciones durante la carga de archivos

Flujo de Ejecución:

1. Usuario hace clic en "Cargar Censo" en el menú de Excel
2. Se abre un diálogo `OpenFileDialog`
3. El archivo seleccionado se abre en Excel mediante `excelApp.Workbooks.Open()`
4. Se instancia `FrmValidaciones` pasando el workbook
5. La ventana se muestra flotante sobre Excel

2. FrmValidaciones.cs - Motor de Validaciones

Este es el corazón del aplicativo. Implementa todas las validaciones de datos y maneja la interacción del usuario.

Variables de Estado Principales:

```
private Excel.Workbook _libroCenso;           // Referencia al archivo cargado
private Excel.Range _rangoCapturado;        // Rango seleccionado para validar
private Excel.Range _rangoCatalogo;         // Rango de opciones para listas
private List<Excel.Range> _pendientesCatalogo; // Rangos acumulados (no usado
actualmente)
private Dictionary<string, string> _preguntaPorRango; // Mapeo pregunta-rango (no
usado actualmente)
```

Constructor:

```
public FrmValidaciones(Excel.Workbook libroAbierto)
{
    _libroCenso = libroAbierto;
    this.TopMost = true;           // Ventana siempre visible
    this.StartPosition = FormStartPosition.CenterScreen;
}
```

Sistema de Validaciones

Tipos de Validaciones Implementadas

1. Validación de Decimales (Enteros)

Objetivo: Detectar y resaltar celdas que contienen números con decimales cuando solo se permiten números enteros.

Implementación Técnica:

- Utiliza **Formato Condicional** de Excel
- Fórmula interna: `=Y(ESNUMERO(A1), TRUNCAR(A1)<>A1)`
- Traduce automáticamente según el idioma del sistema

Comportamiento:

- Comprueba si el valor es un número
- Compara el valor original con su versión truncada
- Si son diferentes, la celda tiene decimales

Formato Aplicado:

- Fondo: Amarillo

- Texto: Rojo, negrita
- Mensajería: Tooltip de información

Código Relevante:

```
private void btnAplicar_Click(object sender, EventArgs e)
{
    if (chkDecimales.Checked == true)
    {
        _rangoCapturado.FormatConditions.Delete(); // Limpiar formatos previos

        string formula = $"=Y(ESNUMERO({direccion}), TRUNCAR({direccion})<>
{direccion})";

        Excel.FormatCondition formato =
(Excel.FormatCondition)_rangoCapturado.FormatConditions.Add(
            Excel.XlFormatConditionType.xlExpression,
            Type.Missing,
            formula);

        formato.Interior.Color = ColorTranslator.ToOle(Color.Yellow);
        formato.Font.Color = ColorTranslator.ToOle(Color.Red);
        formato.Font.Bold = true;
    }
}
```

2. Validación de Catálogos (Listas Desplegables)

Objetivo: Crear listas desplegables (Data Validation) y verificar que los valores estén en minúsculas.

Flujo de Trabajo:

1. Usuario selecciona rango
↓
2. Presiona "Capturar Rango"
↓
3. Marca checkbox "Catálogos"
↓
4. Presiona "Aplicar"
↓
5. Se muestra ventana para seleccionar rango de opciones
↓
6. Usuario selecciona rango de catálogo
↓
7. Se crea lista desplegable y se aplica formato condicional

Componentes Técnicos:

A) Data Validation (Lista Desplegable):

```

_rangoCapturado.Validation.Add(
    (Excel.XlDVType)4,           // Tipo: xlList
    Type.Missing,                // AlertStyle no aplicable
    Type.Missing,                // Operator no aplicable
    refRango,                    // Referencia al rango de opciones
    Type.Missing);               // Formula2 no usada

_rangoCapturado.Validation.IgnoreBlank = true;
_rangoCapturado.Validation.InCellDropdown = true;
_rangoCapturado.Validation.ShowError = true;
_rangoCapturado.Validation.ErrorMessage = "Por favor, seleccione un valor válido de la lista";
_rangoCapturado.Validation.ErrorTitle = "Valor no válido";

```

Consideraciones Importantes:

- `XlDVType` = 4 corresponde a `xlList` en VBA
- No se utilizan `AlertStyle` u `Operator` para listas
- La referencia debe incluir nombre de hoja: `'NombreHoja'!$A$1:$A$10`

B) Formato Condicional (Validación de Minúsculas):

```

string separador =
excelApp.International[Excel.XlApplicationInternational.xlListSeparator].ToString();
string formula2 = $"=Y(ESTEXTO({direccion}){separador} EXACTO({direccion}{separador} MINUSC({direccion}))=FALSO)";

```

```
Excel.FormatCondition formato2 =
(Excel.FormatCondition)_rangoCapturado.FormatConditions.Add(
    Excel.XlFormatConditionType.xlExpression, Type.Missing, formula2);

formato2.Interior.Color = ColorTranslator.ToOle(Color.Yellow);
formato2.Font.Color = ColorTranslator.ToOle(Color.Red);
formato2.Font.Bold = true;
```

Comportamiento:

- Verifica si el valor es texto
- Compara el valor original con su versión en minúsculas
- Si son diferentes, la celda contiene mayúsculas
- Resalta la celda para indicar el problema

Internacionalización:

- Detecta automáticamente el separador de lista del sistema
- La fórmula se construye dinámicamente según el idioma

Detección de Número de Pregunta

Método: `ObtenerNumeroPregunta(Excel.Range rango)`

Este método busca el identificador de la pregunta en la columna A, proporcionando contexto sobre qué pregunta del censo se está validando.

```
private string ObtenerNumeroPregunta(Excel.Range rango)
{
    Excel.Worksheet hoja = rango.Worksheet;
    int filaInicial = rango.Row;

    // Busca hacia arriba desde la fila del rango
    for (int f = filaInicial; f >= 1; f--)
    {
        Excel.Range celdaA = hoja.Cells[f, 1];
        object valor = celdaA.Value2;

        if (valor != null && !string.IsNullOrEmpty(valor.ToString().Trim()))
        {
            return valor.ToString().Trim();
        }
    }
    return "(no encontrada)";
}
```

Lógica:

- Inicia desde la fila del rango seleccionado
- Busca hacia arriba en la columna A
- Retorna el primer valor no vacío encontrado
- Si no encuentra nada, retorna "(no encontrada)"

Flujo de Trabajo

Secuencia Completa de Operación

1. Usuario carga archivo de censo

- Menú SAVCNG > Cargar Censo
- Selecciona archivo .xlsx/.xlsm/.xlsb



2. Se abre ventana FrmValidaciones

- Muestra nombre del archivo cargado
- Ventana siempre por encima de Excel



3. Usuario selecciona rango en Excel

- Marca celdas en el censo
- Presiona "Capturar Rango"



4. Sistema detiene el rango capturado

- Se guarda en _rangoCapturado
- Se detecta número de pregunta
- Se muestra en la interfaz



↓ ↓

Decimales Catálogos

RUTA DECIMALES

5a. Marca checkbox
"Decimales"

6a. Presiona "Aplicar"

RUTA CATÁLOGOS

5b. Marca checkbox
"Catálogos"

6b. Presiona "Aplicar"

```
7a. Aplica formato
condicional
```

```
8a. Muestra resultado
(celdas amarillas)
```

```
7b. Muestra ventana para
seleccionar rango
de opciones
```

```
8b. Usuario selecciona
rango de catálogo
```

```
9b. Se crea lista
desplegable y formato
```

```
10b. Muestra resultado
(lista activa +
formato)
```

Especificaciones Técnicas

Requisitos del Sistema

Aspecto	Requisito
Versión .NET	.NET Framework 4.8
Versión C#	7.3
Versión Excel	2013 o superior
SO	Windows 7/8/10/11
Bitness	32 o 64 bits (según Office instalado)

Dependencias Principales

```
<!-- Librería de integración con Excel -->
Microsoft.Office.Interop.Excel

<!-- Framework ExcelDNA -->
ExcelDna.Integration
ExcelDna.Integration.CustomUI

<!-- Windows Forms (Incluido en .NET Framework) -->
System.Windows.Forms
System.Drawing
```

Manejo de Errores

Estrategia General:

```
try
{
    // Operaciones con Excel
    // Aplicación de validaciones
}
catch (Exception ex)
{
    MessageBox.Show(
        "Ocurrió un error: " + ex.Message,
        "Error",
        MessageBoxButtons.OK,
        MessageBoxIcon.Error);
}
```

Errores Comunes Capturados:

- 0x800A03EC (HRESULT): Parámetros inválidos en validación
- Rango no seleccionado
- Archivo no encontrado
- Acceso denegado

Rendimiento

- **Tiempo de apertura:** < 2 segundos
- **Tiempo de captura de rango:** Inmediato
- **Tiempo de aplicación de validación:** 1-3 segundos (depende del tamaño del rango)
- **Memoria:** ~50-100 MB en uso

Guía de Desarrollo

Estructura de Código

1. Inicialización de Formulario

```
public FrmValidaciones(Excel.Workbook libroAbierto)
{
    InitializeComponent(); // Generado por Visual Studio
    _libroCenso = libroAbierto;
    this.TopMost = true;
    this.StartPosition = FormStartPosition.CenterScreen;
}
```

2. Eventos de Formulario

btnCapturarRango_Click:

- Obtiene la selección actual del usuario
- Valida que sea un rango (no una imagen)
- Guarda en `_rangoCapturado`
- Detecta número de pregunta

btnAplicar_Click:

- Valida que se haya capturado un rango
- Según checkbox marcado, aplica validación
- Maneja excepciones y muestra feedback

chkCatalogos_CheckedChanged:

- Valida precondiciones (archivo cargado, rango capturado)
- Solo marca la casilla (acción real ocurre en btnAplicar)

Mejoras Futuras Posibles

1. **Agregar más tipos de validación:**

- Rango de números (mín/máx)
- Formato de fecha
- Patrón de texto (regex)

2. **Mejorar interfaz:**

- ProgressBar para operaciones largas
- Historial de validaciones
- Exportar reporte de errores

3. **Optimizaciones:**

- Caché de rangos
- Validación por lotes
- Deshacer/Rehacer (Undo/Redo)

4. **Internacionalización:**

- Interfaz en múltiples idiomas
- Soporte para diferentes locales

Puntos de Extensión

Agregar Nueva Validación

1. Crear nuevo método privado:

```
private void AplicarMiValidacion()  
{  
    // Lógica de validación  
}
```

2. Agregar checkbox en el diseñador

3. Agregar lógica condicional en `btnAplicar_Click`:

```
else if (chkMiValidacion.Checked == true)  
{  
    AplicarMiValidacion();  
}
```

Modificar Comportamiento Existente

- **Cambiar colores:** Modificar `ColorTranslator.ToOle(Color.XXX)`
- **Cambiar fórmulas:** Editar strings de fórmula en métodos
- **Cambiar texto:** Actualizar strings en `MessageBox`

Consideraciones de Seguridad

1. Validación de Entrada:

- Se verifica que el rango sea válido antes de operar
- Se valida el tipo de archivo (solo Excel)

2. Gestión de Excepciones:

- Todas las operaciones con Excel están envueltas en try-catch
- Los errores se muestran al usuario

3. Privacidad de Datos:

- El Add-in no envía datos a internet
- Todo ocurre localmente en Excel
- Los archivos no se modifican sin consentimiento

Conclusión

SAVCNG ExcelDNA es un complemento robusto y extensible para validación de datos en Excel. Su arquitectura modular permite fácil mantenimiento y expansión de funcionalidades. El sistema de validación es flexible, soportando desde validaciones simples (decimales) hasta complejas (listas desplegables con formato condicional).

Versión: 1.0.0.0

Última Actualización: 2026

Estado: En Desarrollo